TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

FACE DETECTION WITH RETINAFACE

ĐỒ ÁN MÔN HỌC

Môn: Máy học

TP. HỒ CHÍ MINH, 2026

TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
HO CHI MINH CITY OPEN UNIVERSITY

NHÓM 6

THÀNH VIÊN

2351010176 - Phạm Minh Quân

2351010238 - Lê Khắc Tùng

2351010227 - Phạm Quý Trưởng

2351010242 - Nguyễn Bảo Việt

2351010228 - Hoàng Việt Tuấn

Giáo viên hướng dẫn: LÊ VIẾT TUẤN

TP. HỒ CHÍ MINH, 2026

Mục lục

1 Giới thiệu **1**

1.1 Tổng quan paper: 1

1.1.1 Extra-supervised Learning – Học có giám sát bổ sung: 1

1.1.2 Self-supervised Multi-task Learning – Học đa tác vụ tự giám sát: 2

1.2 WIDER FACE Dataset: 2

2 Metrics và Box trong Face Detection **4**

2.1 Bounding Box: 4

2.2 WIDERFACE Annotations: 5

2.3 Jaccard: 5

2.4 Intersection over Union (IoU): 5

2.5 Average Precision và Mean Average Precision: 6

2.5.1 Average Precision (AP): 6

2.5.2 Mean Average Precision (mAP): 7

3 Related works **8**

3.1 Image Pyramid và Feature Pyramid: 8

3.1.1 Image Pyramid 8

3.1.2 Feature Pyramid 8

3.2 Two-stage và Single-stage: 9

3.2.1 Two-stage: 9

3.2.2 Single-stage: 10

3.3 Context Modelling: (4.2.2) 10

3.4 Multi-task Learning 11

4 RetinaFace Architecture **13**
4.1 Thông tin chung: 13
4.1.1 Multi-task Loss: 13
4.1.2 Dense Regression Branch: 14
4.2 Implementation details: 17
4.2.1 Feature Pyramid 17
4.2.2 Context Module: 20
4.2.3 Loss Head: 23

5 RetinaFace Source Code **24**
5.1 Dataset: 24
5.2 Backbone: 25
5.3 Layers: 27
5.4 Loss: 29
5.5 Optimizer: 31
5.6 Model: 31

6 Ý tưởng cải tiến: **34**
6.1 Hạn chế của paper hiện tại: 34
6.2 Swin Transformer for feature extraction: 34
6.3 Feature Enhancement Feature Pyramid Network: 34

7 Kết quả thực nghiệm: **35**

8 Kết luận **36**

Chương 1. Giới thiệu

1.1 Tổng quan paper:

- Bài toán nhận diện khuôn mặt đã trở thành một trong những bài toán kinh điển trong lĩnh vực **AI for Computer Vision**
- Mặc dù có những bước tiến vượt bậc trong việc phát hiện **uncontrolled face** – phát hiện được những khuôn mặt trong những thách thức về không gian như ánh sáng, pose hay những góc khuất, việc định vị chính xác và hiệu quả khuôn mặt trong môi trường thực tế vẫn đang là thách thức lớn chưa được giải quyết triệt để.
- RetinaFace – Single-stage face detector, được giới thiệu là một giải pháp mạnh mẽ trong việc phát hiện từng **pixel-wise** (điểm ảnh) trên nhiều tỉ lệ khuôn mặt khác nhau bằng cách tận dụng lợi thế của **extra-supervised** và **self-supervised multi-task learning**.

1.1.1 Extra-supervised Learning – Học có giám sát bổ sung:

- Trong Supervised Learning, mô hình được huấn luyện dựa trên bộ dữ liệu có label.
- Trong Extra-supervised Learning, ngoài label có sẵn, ta **cung cấp thêm** cho mô hình các **auxiliary labels (nhãn phụ trợ)** chi tiết hơn ở một khía cạnh khác so với label của tác vụ chính.
- Mục đích:
  + Giảm Overfitting: Việc phải dự đoán đúng nhiều loại label cùng lúc giúp mô hình học được các đặc trưng tổng quát hơn, thay vì chỉ học vẹt một tác

vụ duy nhất.

+ Giúp mô hình chú ý đến các chi tiết quan trọng của dữ liệu mà nếu chỉ sử dụng 1 label chính, mô hình có thể bỏ qua.

1.1.2 Self-supervised Multi-task Learning – Học đa tác vụ tự giám sát:

- **Self-supervised learning (Học tự giám sát):** Là phương pháp học mà mô hình tự tạo ra label từ dữ liệu gốc mà không cần gán label thủ công.
- **Multi-task learning (Học đa tác vụ):** Là phương pháp thiết kế một neural network duy nhất để giải quyết nhiều bài toán khác nhau cùng một lúc (chia sẻ chung các layer trích xuất đặc trưng đầu vào).
- **Self-supervised Multi-task Learning:** Là mô hình học song song nhiều tác vụ cùng một lúc, trong đó có ít nhất một tác vụ tự tạo ra nhãn từ input.
- Mục đích:
  + Tận dụng tối đã dữ liệu chưa gán nhãn: Cho phép mô hình học các biểu diễn hình học, cấu trúc hoặc ngữ cảnh thông qua nhánh self-supervised, từ đó hỗ trợ tăng cường độ chính xác cho các nhánh học có giám sát khác.

1.2 WIDER FACE Dataset:

- RetinaFace sử dụng **WIDER FACE** dataset – **Face Detection Benchmark** [1].
- Bộ dataset này chứa nhiều annotations khác nhau bao gồm: Occlusions (độ che khuất), poses, event categories, face bounding boxes...
- WIDER FACE bao gồm 32,203 ảnh và 393,703 face bounding boxes với mức độ biến đổi cao về scale, poses, expression, occlusion và illumination.
- WIDER FACE được chia thành 40% training, 10% validation và 50% testing bằng cách **randomly sampling** (Lấy mẫu ngẫu nhiên) từ 61 scene categories.

– Dựa vào tỷ lệ phát hiện của thuật toán EdgeBox, khuôn mặt được phân thành 3 levels: Easy, Medium và Hard thông qua việc bổ sung dần các mẫu khó.

– **Extra Annotations.** Nhóm tác giả paper đã xác định được 5 cấp độ chất lượng hình ảnh khuôn mặt (levels of face image quality) và tiến hành gán nhãn thủ công, bổ sung 5 điểm mốc khuôn mặt (facial landmarks) bao gồm: tâm 2 mắt, chóp mũi và 2 khóe miệng.

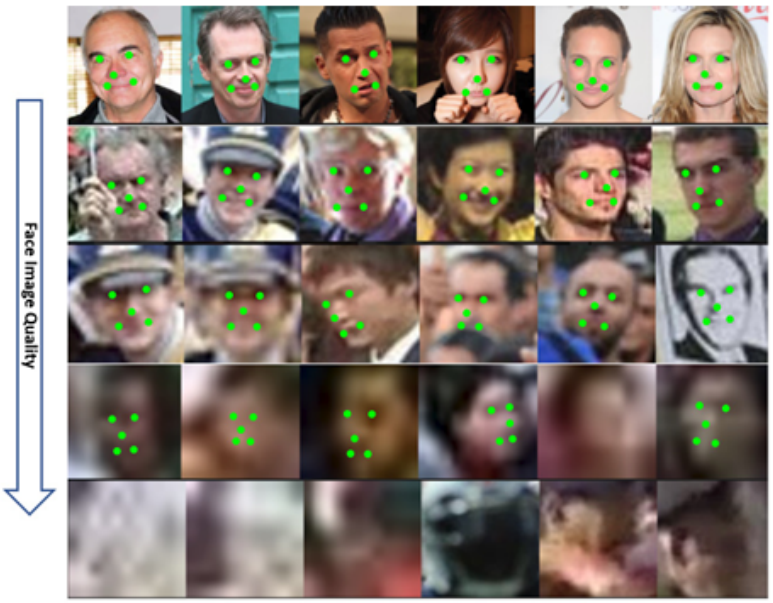


Figure 1.1: Các extra annotations của 5 facial landmarks

– Tổng cộng đã gán nhãn 84,600 faces trên training set và 18,500 faces trên validation set

Chương 2. Metrics và Box trong Face Detection

2.1 Bounding Box:

– Bounding box là phần khung bao quanh 1 object.

– **Các chuẩn biểu diễn tọa độ phổ biến:**

+ Min max: $(x_{min}, y_{min}, x_{max}, y_{max})$

+ Center-Size: $(x_{center}, y_{center}, width, height)$

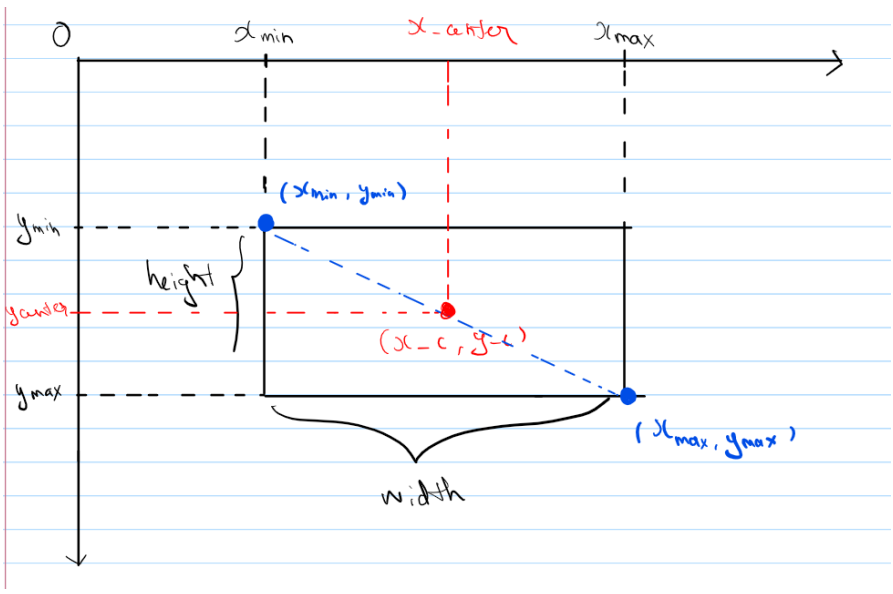


Figure 2.1: Hệ trục tọa độ của các điểm trên

2.2 WIDERFACE Annotations:

– **Cấu trúc annotations** trong label.txt mà paper cung cấp:

+ File name

+ Number of bounding Box

+ x_1, y_1, w, h, blur, expression, illumination, invalid, occlusion, pose

2.3 Jaccard:

– **Jaccard Index (Hệ số tương đồng Jaccard)** – là một con số dùng để đo lường mức độ tương đồng giữa hai tập hợp dữ liệu trong xác suất thống kê (statistic)

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

2.4 Intersection over Union (IoU):

– **IoU** là một metric dùng để đánh giá mức độ trùng lấp (overlap) giữa 2 bounding boxes.

– Công thức tổng quát:

$$\textbf{IoU} = \frac{\textbf{Area of Intersection}}{\textbf{Area of Union}}$$

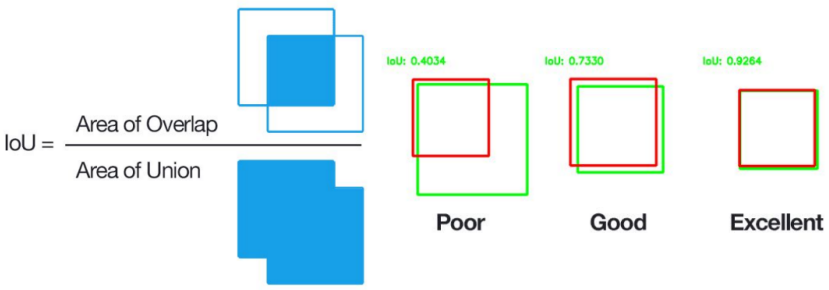


Figure 2.2: Minh họa IoU

Intersection: Phần giao giữa hai bounding boxes

Union: Phần hợp giữa hai bounding boxes

- Hai bounding boxes càng overlap nhau, IoU càng **gần 1**, độ chính xác càng cao, object được detect càng tốt.
- Hai bounding boxes càng lệch nhau, IoU càng **gần 0**, độ chính xác càng giảm, object detect sai

2.5 Average Precision và Mean Average Precision:

2.5.1 Average Precision (AP):

- AP dùng để đo độ chính xác của mô hình cho một class duy nhất.
- Cách tính:
  + Liệt kê tất cả các bounding boxes mà mô hình dự đoán được, sắp xếp giảm dần dựa vào **confidence score**
  + So sánh với ground truth bằng IoU qua threshold (Ngưỡng), thường là 0.5.
    . **True Positive (TP):** Dự đoán có IoU $\geq$ threshold với Ground Truth
    . **False Positive (FP):** Dự đoán có IoU < threshold với Ground Truth
    . **False Negative FN:** Ground truth mà mô hình bỏ lỡ (không detect được)
  + Tính Precision và Recall:

$$Precision = \frac{TP}{TP+FP} \quad ; \quad Recall = \frac{TP}{TP+FN}$$

  + Vẽ Precision-Recall Curve:

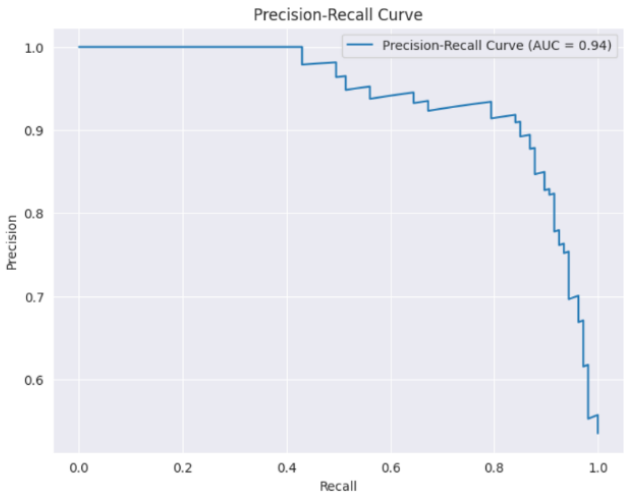


Figure 2.3: Minh họa Precision-Recall Curve

+ AP = diện tích dưới đường Precision-Recall (Area under the precision-recall curve AUC-PR)
+ Công thức tổng quát:

$$AUCPR = \int_0^1 P(R)\,dR$$

. $P(R)$: precision tại recall R

+ Công thức rời rạc:

$$AUCPR = \sum_n (R_n - R_{n-1})P_n$$

. P_n: Precision tại điểm n

. R_n: Recall tại điểm n

. $R_n - R_{n-1}$: Độ tăng recall (R_n và R_{n-1} là các mức Recall kế tiếp nhau sau khi đã sắp xếp theo Confidence Score)

2.5.2 Mean Average Precision (mAP):

– mAP là trung bình giá trị AP của tất cả classes.

$$mAP = \frac{1}{N}\sum_{i=1}^{N} AP_i$$

+ N: Số lượng classes.
+ AP_i: giá trị AP tại class thứ i.

Chương 3. Related works

3.1 Image Pyramid và Feature Pyramid:

Image Pyramid và Feature Pyramid là hai phương pháp được sử dụng để giải quyết thách thức về sự thay đổi tỷ lệ (scale) kích thước khuôn mặt.

3.1.1 Image Pyramid

- Mô hình dựa trên cơ chế sliding-windows, trong đó classifier sẽ được quét qua một lưới hình ảnh dày đặc (dense image grid)

3.1.2 Feature Pyramid

- Mô hình được xây dựng dựa trên **Image Pyramid**, kết hợp giữa sliding-anchor (cơ chế mỏ neo trượt) trên multi-scale feature maps đã thống trị **(dominated)** trong lĩnh vực face detection.

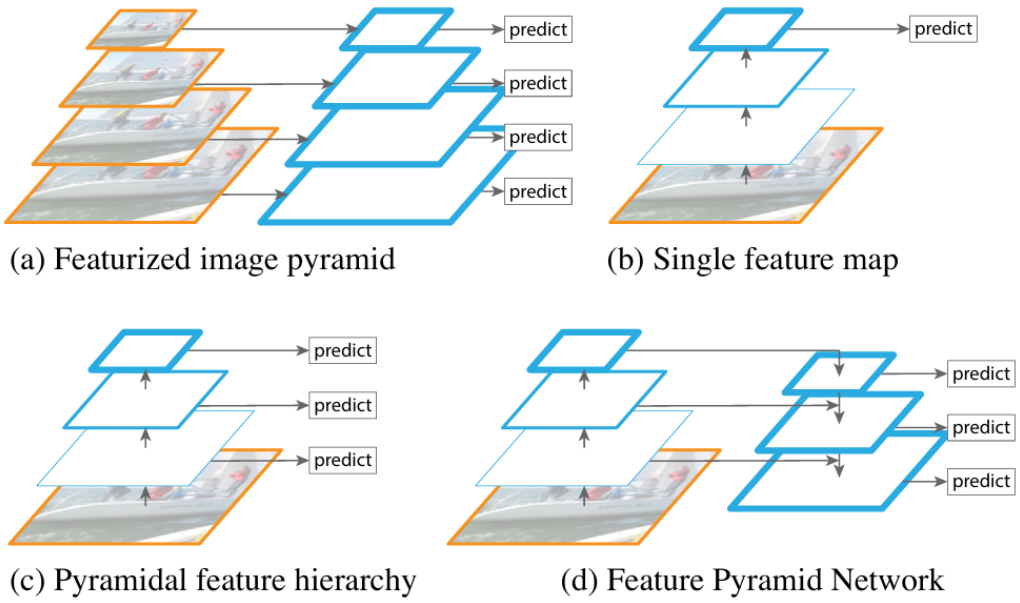


Figure 3.1: Minh họa Feature Pyramid được xây dựng dựa trên Image Pyramid

3.2 Two-stage và Single-stage:

3.2.1 Two-stage:

- **Giai đoạn 1 - Đề xuất vùng (Region Proposal Generation):** Sử dụng các thuật toán như Selective Search hoặc Region Proposal Network (RPN) trong các kiến trúc hiện đại (Faster R-CNN) để xác định các vùng ứng viên có khả năng chứa object.

- **Giai đoạn 2 - Phân loại và Tinh chỉnh (Classification & Regression):** Với mỗi Region Proposal, mô hình thực hiện đồng thời hai nhiệm vụ:

  - *Phân loại đối tượng (Object Classification):* Phân loại đối tượng trong region proposal thuộc class nào.

  - *Hồi quy khung hình (Bounding Box Regression):* Tính toán các độ sai lệch (offsets) để điều chỉnh tọa độ của vùng đề xuất ban đầu sao cho (fit) object nhất.

3.2.2 Single-stage:

- Object Classification và Bounding Box Regression được thực hiện trực tiếp trên toàn bộ image mà không cần qua Region Proposal Generation.
- RetinaFace là mô hình kiến trúc Single-Stage.

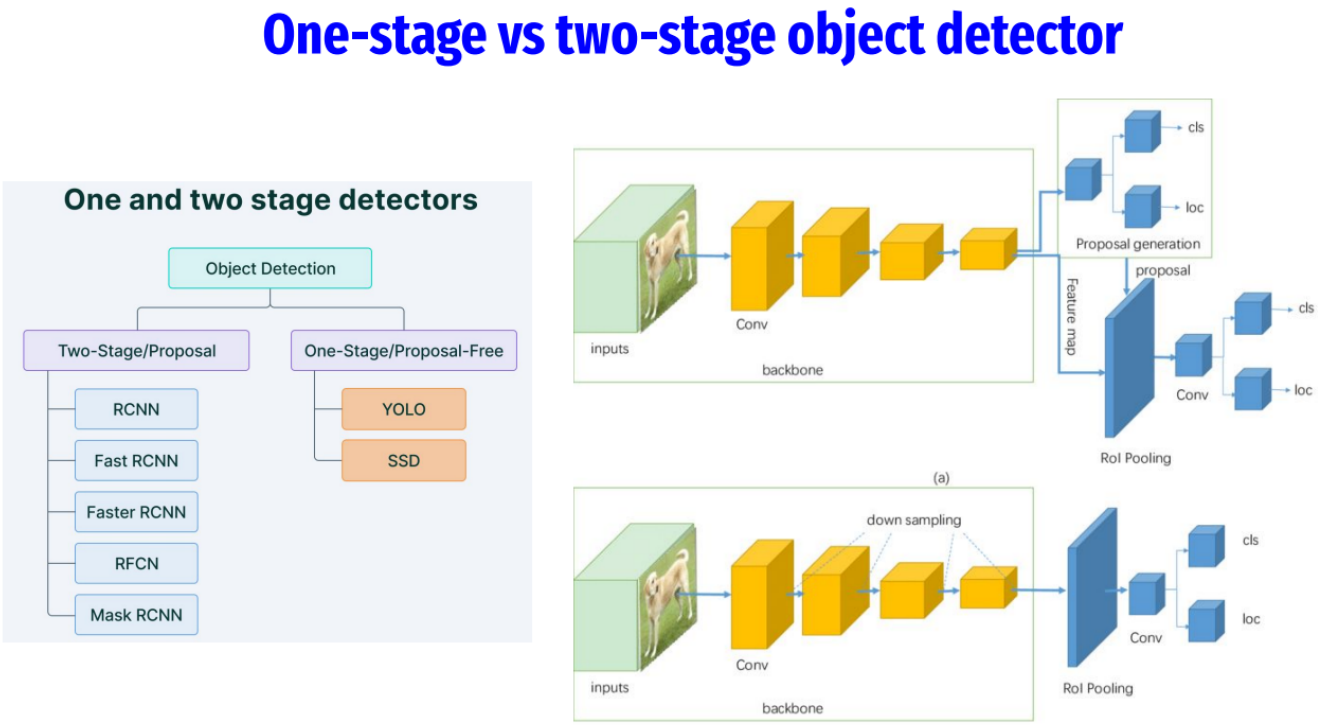


Figure 3.2: Minh họa Single-stage và Two-stage Object Detector

3.3 Context Modelling: (4.2.2)

- Context Modelling là một kỹ thuật để tăng cường khả năng suy luận ngữ cảnh của neural network, đặc biệt nhằm mục đích phát hiện các khuôn mặt cực kỳ nhỏ (tiny face).
- Trong RetinaFace, để tăng cường khả năng capturing tiny faces, SSH **(Single Stage Headless) - Module for feature extraction** và PyramidBox đã áp dụng context modules trên feature pyramid để mở rộng receptive field từ Euclidean grids.

- Để tăng cường non-rigid transformation modelling, RetinaFace sử dụng **Deformable Convolution Network (DCN)** [2] để thay thế toàn bộ 3 x 3 convolution layers bên trong lateral connections cũng như bên trong chính các context modelling.

3.4 Multi-task Learning

- Là cách học mà một mô hình có thể giải quyết được nhiều tasks khác nhau
- Trong RetinaFace, Multi-task Learning là chiến lược thiết kế một neural network duy nhất để giải quyết đồng thời nhiều bài toán face detection khác nhau từ cùng một input image, giúp các bài toán chia sẻ feature extraction và bổ trợ chéo cho nhau.
- Cấu trúc Multitask-Learning trong RetinaFace: sử dụng một lớp tính toán dùng chung (shared loss head với 1 x 1 convolution layer) áp dụng lên các tầng của feature pyramid. Với mỗi positive anchor, network sẽ dự đoán đồng thời 4 thông tin:
  + Face score
  + Face box
  + 5 facial landmarks
  + Dense 3D face vertices: Cấu trúc 3D dày đặc của khuôn mặt được chiếu lên image plane (self-supervised Learning).

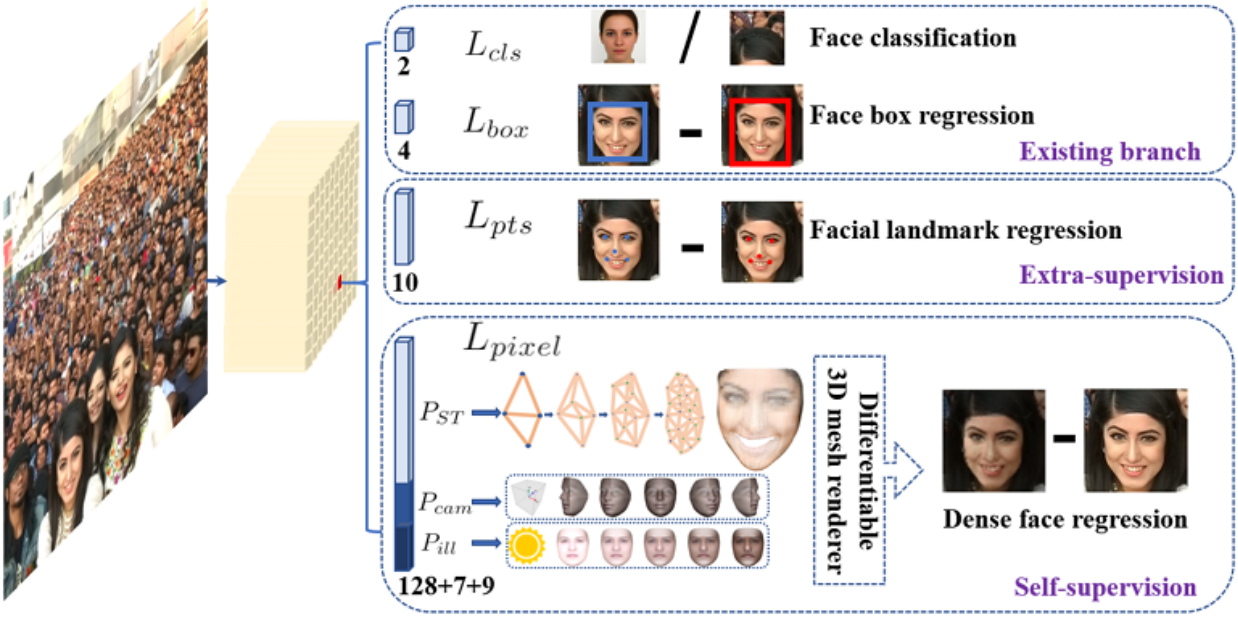


Figure 3.3: Phương pháp single-stage pixel-wise được đề xuất bằng cách sử dụng extra-supervised và self-supervised multi-task learning song song với các box classification và regression branches hiện có. Mỗi positive anchor bao gồm: (1) face score, (2) face box, (3) 5 facial landmarks, (4) dense 3D face vertices được chiếu trên image plane

Chương 4. RetinaFace Architecture

4.1 Thông tin chung:

4.1.1 Multi-task Loss:

- Với mỗi training anchor i, ta sẽ minimise multi-task loss có công thức:

$$L = L_{cls}(p_i, p_i^*) + \lambda_1 p_i^* L_{box}(t_i, t_i^*) + \lambda_2 p_i^* L_{pts}(l_i, l_i^*) + \lambda_3 p_i^* L_{pixel}. \quad (1)$$

- (1) Face classification loss $L_{cls}(p_i, p_i^*)$: Là **Softmax loss for binary classes (face/not face)** dùng để tính xác suất p_i xem anchor thứ i có phải là face hay không, $p_i^* = 1$ nếu positive anchor (có chứa face) và 0 nếu negative anchor

- (2) Face box regression loss $L_{box}(t_i, t_i^*)$ Là regression loss:

  + Với $t_i = \{t_x, t_y, t_w, t_h\}_i$ và $t_i^* = \{t_x^*, t_y^*, t_w^*, t_h^*\}_i$ lần lượt là tọa độ (coordinates) của predicted box và grouth truth box liên quan đến positive anchor $(p_i^* = 1)$

  + Paper tuân thủ theo [3] để normalise box regression targets (centre location, width, height) và $L_{box}(t_i, t_i^*) = R(t_i - t_i^*)$ với R là robust loss function (smooth-L1) được định nghĩa trong [3].

$$Smooth_{L1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise,} \end{cases} \quad (1.1)$$

- (3) Facial landmark regression Loss $L_{pts}(l_i, l_i^*)$

  + Với $l_i = \{l_{x_1}, l_{y_1}, \ldots, l_{x_5}, l_{y_5}\}_i$ và $l_i = \{l_{x_1}^*, l_{y_1}^*, \ldots, l_{x_5}^*, l_{y_5}^*\}_i$ lần lượt là predicted 5 facial landmarks và grouth truth liên quan đến positive anchor.

+ Tương tự box centre regression, 5 facial landmark regression cũng sử dụng target normalisation dựa trên anchor centre.

– (4) Dense regression loss L_{pixel} (tham chiếu mục (4.1.2))

– Loss-balancing parameters $\lambda_1 - \lambda_3$ được gán giá trị lần lượt là 0.25, 0.1 và 0.1. Có nghĩa là ta tăng đáng kể tầm quan trọng của box và landmark locations từ supervision signals.

4.1.2 Dense Regression Branch:

– **Mesh Decoder:** paper đã employ trực tiếp mesh decoder (mesh convolution và mesh up-sampling) từ [4, 5].

+ Thay vì sử dụng convolution 2D thông thường (tính toán trên lưới không gian Euclid), nhánh này sử dụng **Graph Convolution** (tích chập đồ thị) để thao tác với các đỉnh trên mô hình lưới 3D.

+ Mesh decoder thực hiện dự đoán đồng thời cả shape và texture của face thông qua phương pháp fast localised spectral filtering (lọc quang phổ cục bộ nhanh).

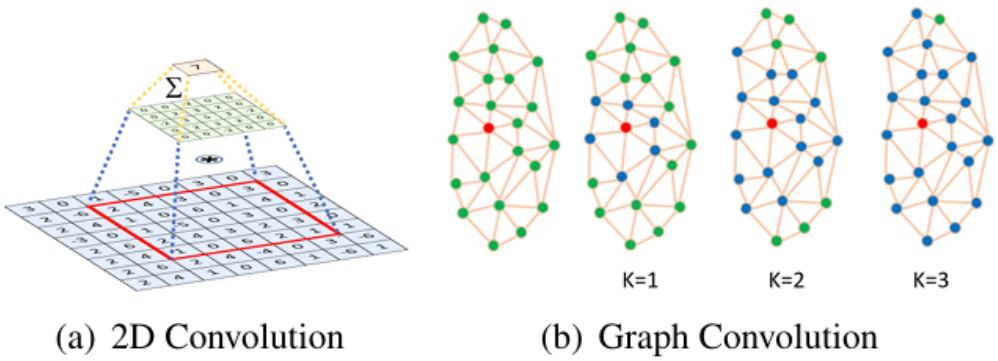


Figure 4.1: Minh họa 2D Convolution trong không gian lưới Euclid và Graph Convolution

∗ **(a) 2D Convolution** là một *kernel-weighted neighbour sum* bên trong lưới Euclid receptive field. Mỗi lớp tích chập (*convolutional layer*)

chứa $\text{Kernel}_H \times \text{Kernel}_W \times \text{Channel}_{in} \times \text{Channel}_{out}$ tham số.

* **(b) Graph Convolution** cũng là một dạng của *kernel-weighted neighbour sum*, nhưng khoảng cách lân cận (*neighbour distance*) được tính trên đồ thị bằng cách đếm số cạnh tối thiểu nối giữa hai đỉnh. Mỗi lớp tích chập đồ thị có $K \times \text{Channel}_{in} \times \text{Channel}_{out}$ tham số và các hệ số Chebyshev $\theta_{i,j} \in \mathbb{R}^K$ được cắt bớt (*truncated*) ở bậc K.

+ RetinaFace tuân thủ theo [4] để định nghĩa coloured face mesh $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, với $\mathcal{V} \in R^{n\times 6}$ là 1 set của face vertices chứa thông tin về hình dạng (shape) và kết cấu (texture) của khớp (joint), và $\mathcal{E} \in \{0,1\}^{n\times n}$ là ma trận kề **thưa** (sparse adjacency matrix) encoding trạng thái kết nối giữa các vertices.

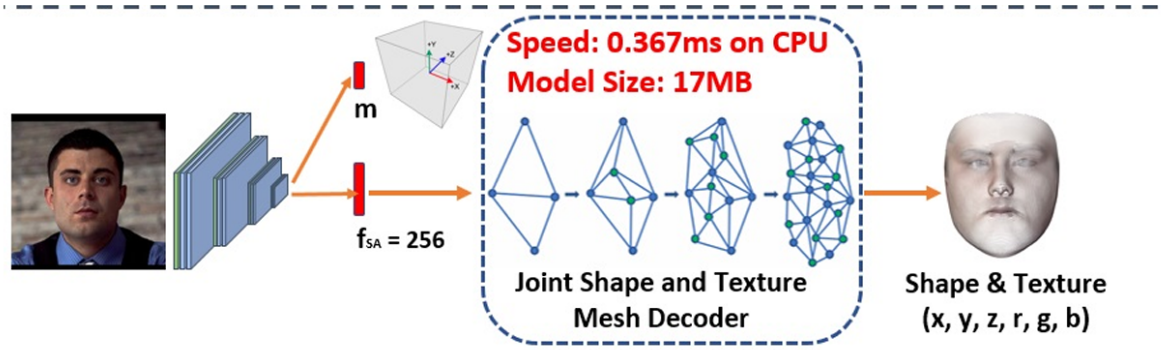


Figure 4.2: Minh họa dataflow của mesh decoder mà paper lấy ý tưởng. Image sẽ được đưa qua feature pyramid để tạo ra các feature maps, tại Loss Head, một lớp 1 x 1 convolution layer dùng để trích xuất feature vector duy nhất cho mỗi anchor, từ vector này network predict ra các tham số chung cho cả texture và shape $P_st \in \mathbb{R}^{128}$. Tiếp theo tham số được đưa qua Joint Mesh Decoder (sử dụng graph convolution) để tạo ra lưới $V \in \mathbb{R}^{n\times 6}$ chứa cả Shape và Texture trên cùng một vertice.

+ Đồ thị Laplacian được định nghĩa $\mathcal{L} = \mathcal{D} - \mathcal{E} \in \mathbb{R}^{n\times n}$ với $\mathcal{D} \in \mathbb{R}^{n\times n}$ là ma trận đường chéo (diagonal matrix) với $\mathcal{D}_{ii} = \sum_j \mathcal{E}_{ij}$

+ Thông qua [4–6], graph convolution với kernel g_θ có thể biểu diễn như recursive Chebyshev polynominal (đa thức Chebyshev đệ quy) được truncated ở bậc K

$$y = g_\theta(L)x = \sum_{k=0}^{K-1} \theta_k T_k(\tilde{L})x, \tag{2}$$

+ Với $\theta \in \mathbb{R}^K$ là vector của hệ số Chebyshev và $T_k(\tilde{K} \in \mathbb{R}^{n \times n})$ là đa thức Chebyshev tại bậc k được evaluated tại scale Laplacian $\tilde{L}$

+ Với $\bar{x} = T_k(\tilde{L})x \in \mathbb{R}^n$, ta có thể tính đệ quy $\bar{x}_k = 2\tilde{K}\bar{x}_{k-1} - \bar{x}_{k-2}$ với $\bar{x}_0 = x$ và $\bar{x}_1 = \tilde{L}x$

+ Toàn bộ quá trình lọc (filtering operation) cực kỳ hiệu quả bao gồm K tích ma trận thưa với một tích ma trận dày đặc **(dense matrix-vector)** $y = g_\theta(L)x = [\bar{x}_0, \ldots, \bar{x}_{k-1}]\theta \rightarrow$ **Fast localised spectral filtering**

– **Differentiable Renderer:** Sau khi predict được shape và texture parameters $P_{ST} \in \mathbb{R}^{128}$, nhóm tác giả đã sử dụng một bộ kết xuất lưới 3D khả vi (efficient differentiable 3D mesh renderer) để chiếu coloured-mesh (bộ lưới màu) $\mathcal{D}_{P_{ST}}$ lên trên một mặt phẳng 2D image với các camera parameters $P_{cam} = [x_c, y_c, z_c, x'_c, y'_c, z'_c, f_c]$ (camera location, camera pose và focal length (tiêu cực)) và illumination parameters (tham số ánh sáng | tham số chiếu sáng) $P_{ill} = [x_l, y_l, z_l, r_l, g_l, b_l, r_a, g_a, b_a]$ (location of point light source (vị trí của điểm của nguồn sáng), colour values (giá trị màu) và colour of ambient lighting (màu sắc của ánh sáng môi trường)).

– **Dense Regression Loss:** Sau khi có được rendered 2D face $\mathcal{R}(\mathcal{D}_{P_{ST}}, P_{cam}, P_{ill})$, ta sẽ tính toán sự khác biệt giữa pixel-wise của ảnh rendered và ảnh 2D gốc (được cắt ra từ anchor) sử dụng L_{pixel}:

$$L_{pixel} = \frac{1}{W * H} \sum_{i}^{W} \sum_{j}^{H} \left\| \mathcal{R}(\mathcal{D}_{P_{ST}}, P_{cam}, P_{ill})_{i,j} - I^*_{i,j} \right\|_1 \tag{3}$$

+ Với W và H là width và height của anchor crop $I^*_{i,j}$

+ Anchor crop $I^*_{i,j}$ chính là original 2D face được cắt ra (crop) từ anchor.

+ Hàm loss này sử dụng L1-norm để tính sai số tuyệt đối, sau đó chia trung bình cho tổng điểm số ảnh $(W * H)$

+ L1-norm: Là tổng giá trị tuyệt đối của các phần tử trong vector (vector's components) $\rightarrow$ Manhattan norm:

$$\|x\|_1 = \sum_{i=1}^{n} |x_i|$$

4.2 Implementation details:

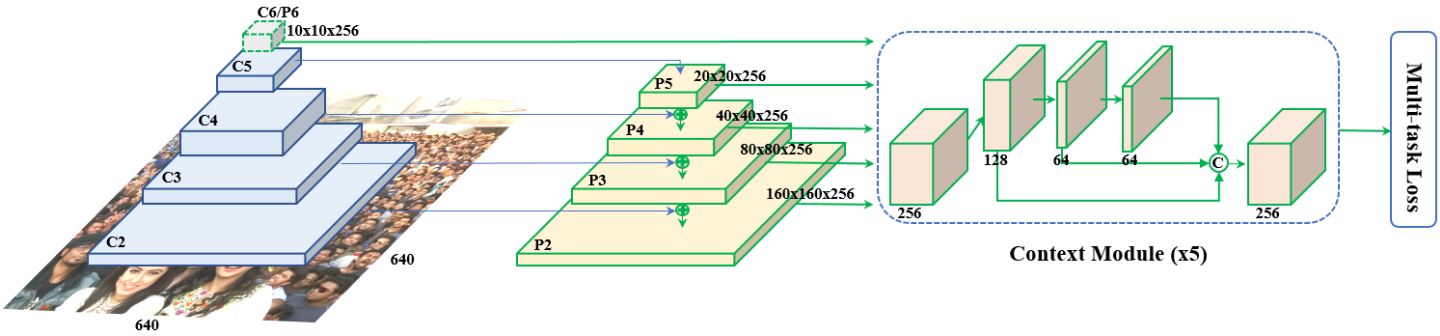


Figure 4.3: Overview về phương pháp single-stage dense face localisation. RetinaFace được xây dựng dựa trên feature pyramids với những context module độc lập. Qua context modules, ta tính multi-task loss cho mỗi anchor

4.2.1 Feature Pyramid

- RetinaFace đã sử dụng feature pyramid levels từ $P_2 \rightarrow P_6$, với $P_2 \rightarrow P5$ được tính từ output (features map) tương ứng ResNet (Residual Network) $(C_2$ đến $C_5)$ sử dụng top-down (kết nối từ trên xuống) và lateral connections (kết nối ngang).
- P_6 được tính thông qua 3×3 convolution với stride = 2 trên C_5
- $C_1 \rightarrow C_5$ sử dụng backbone là pre-trained ResNet-152 [7] classification network trên bộ ImageNet-11K trong khi P_6 được khởi tạo ngẫu nhiên bằng phương pháp "Xavier" [8]

1. **Xavier method:**
   + Một phương pháp do nhóm tác giả của Xavier Glorot giới thiệu vào năm 2010.
   + Đây là phương pháp khởi tạo trọng số ban đầu (initial weights) cho các neural network.
   + Trong quá trình training deep neural network, nếu initial weights quá bé, dẫn đến **Vanishing Gradient**. Nếu initial weights quá lớn có thể dẫn đến

Exploding Gradient. Xavier method được sinh ra nhằm giữ cho phương sai (variance) của các input và output variance ở mỗi lớp xấp xỉ bằng nhau, giúp các tín hiệu luân chuyển ổn định xuyên suốt network.

+ Trọng số W của một lớp neural sẽ được gán ngẫu nhiên bằng cách lấy mẫu từ một phân phối (thường là Uniform distribution (Phân phối đều) hoặc Normal distribution (Phân phối chuẩn)) có mean = 0 và variance được định nghĩa là:

+ **Uniform Xavier Initialization:**

$$x = \sqrt{\frac{6}{n_{inputs} + n_{outputs}}} \tag{4.1}$$

  – n_{inputs}: Số lượng input của input layer
  – $n_{outputs}$: Số lượng output của output layer
  – W sẽ được gán ngẫu nhiên trong uniform distribution giá trị nằm trong khoảng $[-x, x]$

+ **Normal Xavier Initialization**:

$$\sigma = \sqrt{\frac{2}{n_{inputs} + n_{outputs}}}$$

  – n_{inputs}: Số lượng input của input layer
  – $n_{outputs}$: Số lượng output của output layer
  – W sẽ được gán ngẫu nhiên trong normal distribution với mean = 0 và standard deviation (std) σ

$$W \sim \mathcal{N}(0, \sigma^2)$$

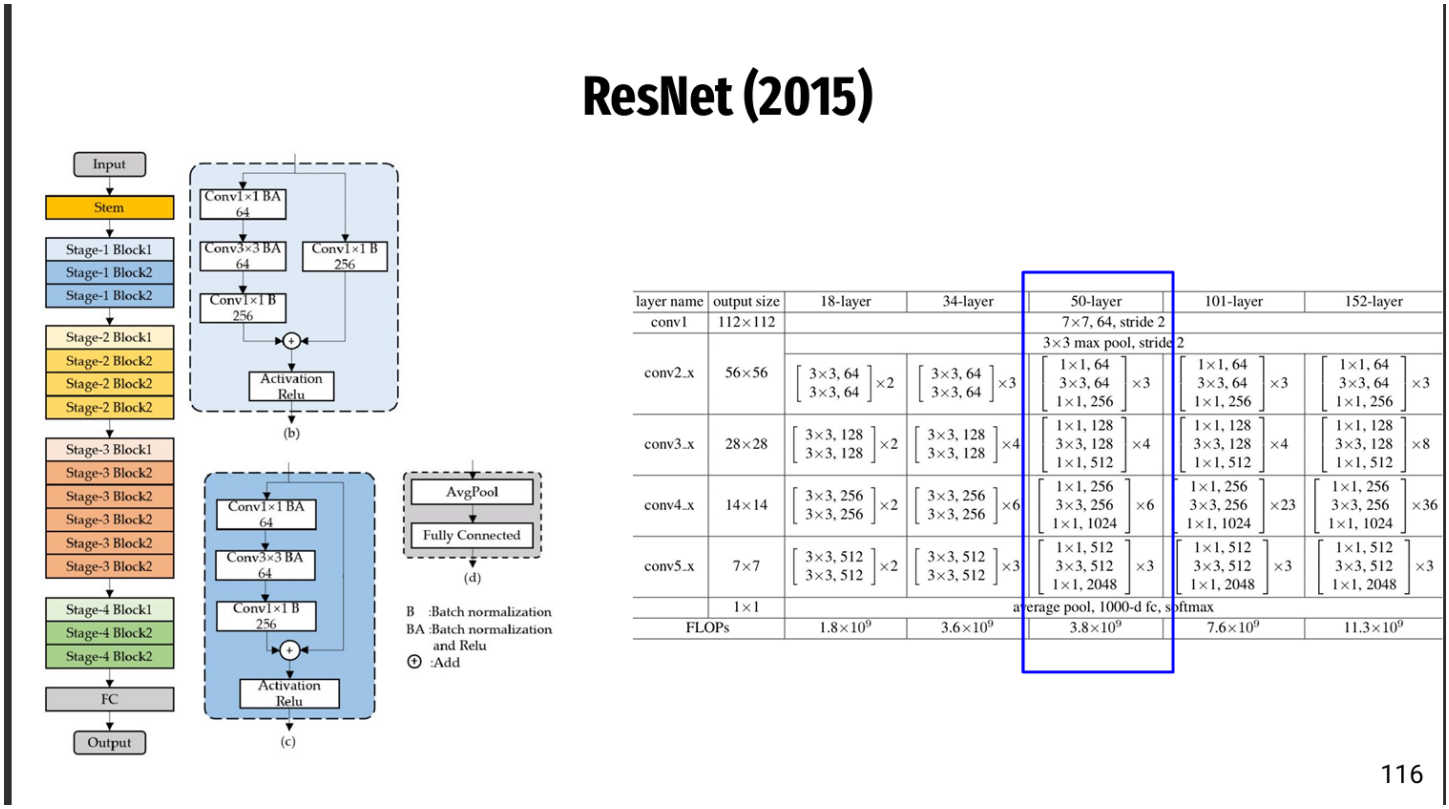
2. **ResNet:** [7]

+ **ResNet (Residual Network)** được giới thiệu vào 2015 – 2016, đây là một kiến trúc CNN.

+ Khi neural network có kiến trúc càng sâu (nhiều layer), hiện tượng Vanishing Gradient rất dễ xảy ra. ResNet sẽ giúp giải quyết vấn đề này bằng Residual Learning, bằng cách sử dụng các Residual Blocks kết hợp với Skip Connection, thay vì học trực tiếp hàm $H(x)$, mạng học phần dư:

$$F(x) = H(x) - x \quad ; \quad H(x) = F(x) + x$$

+ Phần x được truyền trực tiếp qua skip connection, giúp gradient truyền tốt hơn trong backpropagation.



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	[3×3, 64; 3×3, 64] ×2	[3×3, 64; 3×3, 64] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3	[1×1, 64; 3×3, 64; 1×1, 256] ×3
conv3_x	28×28	[3×3, 128; 3×3, 128] ×2	[3×3, 128; 3×3, 128] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×4	[1×1, 128; 3×3, 128; 1×1, 512] ×8
conv4_x	14×14	[3×3, 256; 3×3, 256] ×2	[3×3, 256; 3×3, 256] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×6	[1×1, 256; 3×3, 256; 1×1, 1024] ×23	[1×1, 256; 3×3, 256; 1×1, 1024] ×36
conv5_x	7×7	[3×3, 512; 3×3, 512] ×2	[3×3, 512; 3×3, 512] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3	[1×1, 512; 3×3, 512; 1×1, 2048] ×3
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Figure 4.4: Minh họa ResNet, ô được tô xanh là model ResNet50, ResNet152 là 152-layer bên phải.

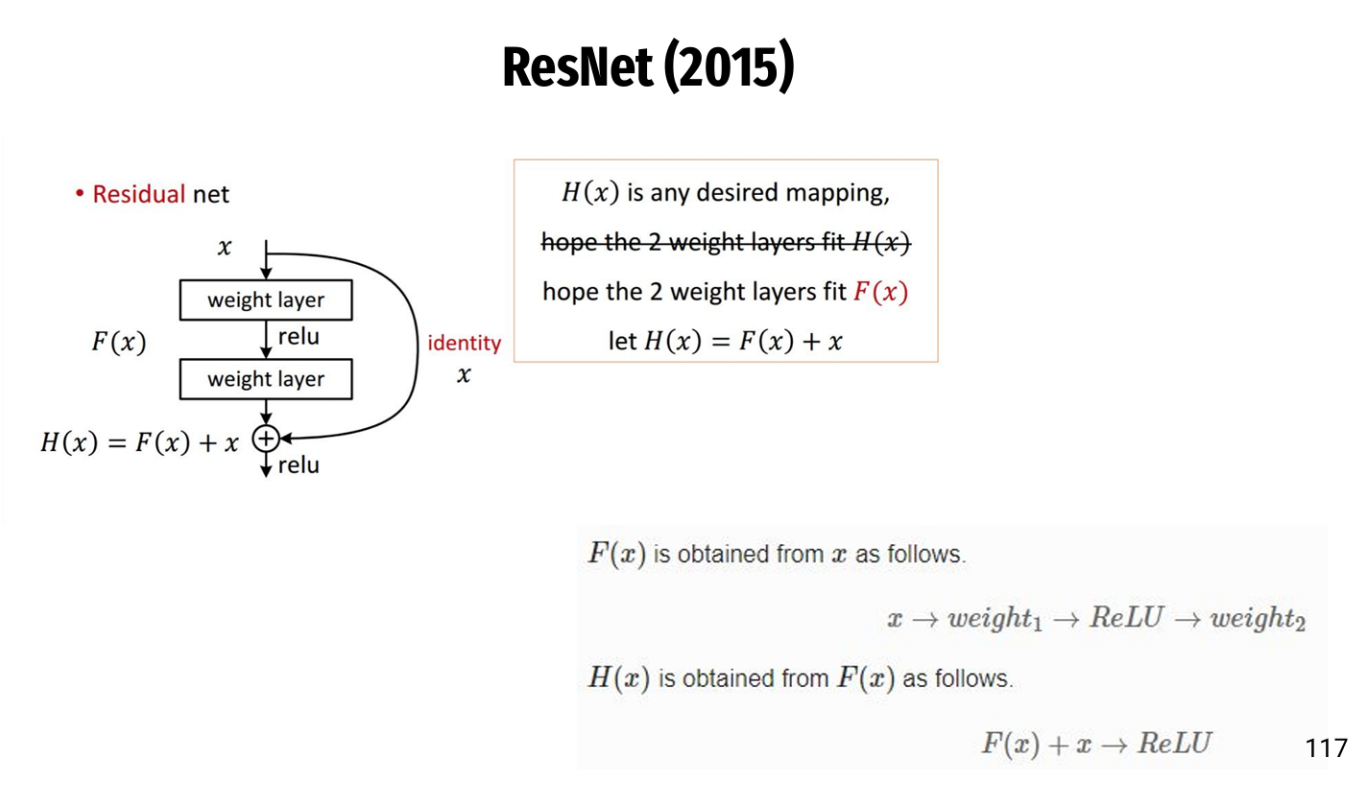


Figure 4.5: Minh họa cách residual learning hoạt động

4.2.2 Context Module:

– Được lấy cảm hứng từ SSH và PyramidBox, RetinaFace cũng áp dụng những context modules độc lập lên 5 tầng feature pyramid để mở rộng receptive field và tăng cường khả năng của rigid context modelling

– Mô hình đã thay thế toàn bộ 3×3 convolution layers bên trong lateral connection và context modules thành các deformable convolution network (DCN), giúp tăng cường non-rigid context modelling.

1. **SSH và PyramidBox:**

   + **SSH (Single Stage Headless Face Detector):**

     – **Headless:** Kiến trúc này loại bỏ hoài toàn fully connected layers, giúp model nhẹ và nhanh hơn.

     – **Context Module:** Thay vì dùng các filter có kích thước lớn (như 5×5 hay 7×7) tốn nhiều chi phí để mở rộng receptive field, SSH kết hợp

song song các 3×3 convolution layers. Việc stack hai lớp 3×3 sẽ tạo hiệu ứng giống một lớp 5×5, xếp chồng ba lớp 3×3 tạo hiệu ứng 7×7. Output của các nhanh này được concatenate lại với nhau, giúp network vừa nhẹ vừa bao quát được ngữ cảnh xung quanh khuôn mặt.

+ **PyramidBox (Context-assisted Single Shot Face Detector)**: Phức tạp hơn và chịu trách nhiệm xử lý các khuôn mặt bị che khuất hoặc tiny face, bao gồm:
  - **Low-level Feature Pyramid Network**: Kết hợp thông tin từ nhiều layer của VGG16 network để tạo feature maps có độ phân giải cao, phù hợp cho tiny face.
  - **CPM (Context-sensitive Predictor Module):** CPM predict thêm các bộ phận (đầu, vai, cơ thể) thay vì chỉ bounding box của khuôn mặt, giúp network có thể suy luận ra được vị trí khuôn mặt ngay cả khi bị mờ hoặc bị khuất.

2. **DCN (Deformable Convolutional Networks - Mạng tích chập biến dạng):** [2]

+ Trong CNN cổ điển, các filter luôn sử dụng 3×3 hay 5×5. Tuy nhiên, trong thực tế khuôn mặt người có nhiều phức tạp (pose, emotion, ...) khiến khả năng detect gặp nhiều hạn chế:

$$y(p_0) = \sum_{p_n \in \mathcal{R}} w(p_n) \cdot x(p_0 + p_n)$$

  - Với $\mathcal{R}$ là regular gird, là tập hợp các spatial locations xác định vị trí mà fitler "nhìn" vào để trích xuất dữ liệu trong feature map. Xác định receptive field size và dilation (độ giãn):

$$\mathcal{R} = \{(-1,-1),(-1,0),\ldots,(0,1),(1,1)\}$$

    xác định một kernel 3×3 với dilation 1
  - w là trọng số của filter, x là giá trị pixel đầu vào và p_n là các vị trí cố định trên grid. Lưới này là cố định, do đó sẽ tiếp nhận cả pixel nhiễu ở background nếu object không có hình vuông.

+ DCN giúp giải quyết vấn đề trên. Cấu trúc của nó bao gồm 2 bước hoạt động song song:
  - **Nhánh học độ lệch (Offset Learning):** Feature map sẽ được đưa qua 1 lớp convolution để dự đoán các giá trị độ lệch $\{\Delta p_n | n = 1, \ldots, N\}$, với $N = |\mathcal{R}|$. Các giá trị độ lệch này là số thực (ví dụ: lệch 1.5 pixel sang trái, 0.2 pixel xuống dưới).
  - **Nhánh tích chập biến dạng (deformable convolution):** Convolution layer sẽ lấy mẫu ở vị trí đã được cộng thêm độ lệch thay vì lưới vuông cố định:

$$y(p_0) = \sum_{p_n \in \mathcal{R}} w(p_n) \cdot x(p_0 + p_n + \Delta p_n)$$

  - Vì vị trí mới $p_0 + p_n + \Delta p_n$ thường là số thập phân và không nằm tròn trên một pixel cụ thể, DCN sử dụng bilinear interpolation (nội suy song tuyến tính) để tính toán giá trị tại điểm đó dựa trên 4 pixel gần nhất.

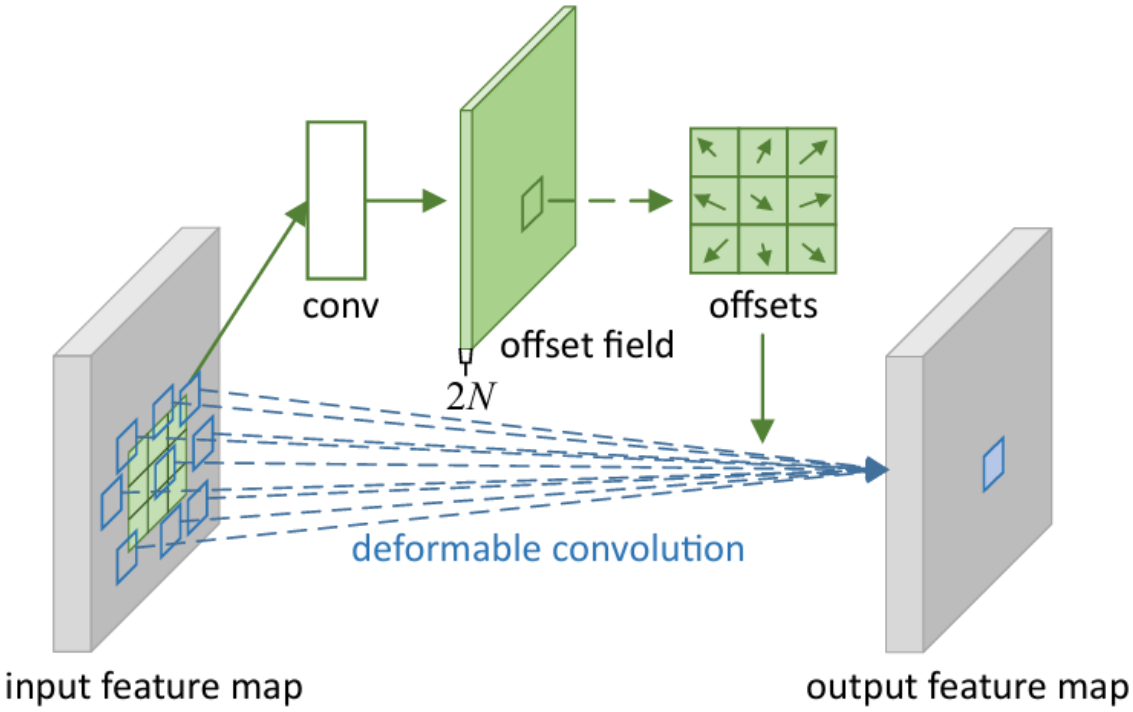


Figure 4.6: Minh họa 3×3 deformable convolution.

4.2.3 Loss Head:

- Với negative anchors, chỉ áp dụng classification loss.
- Với positive anchors, sử dụng multi-task loss.
- RetinaFace triển khai shared loss head (1×1 conv) xuyên suốt feature maps ở các tầng khác nhau $H_n \times W_n \times 256, n \in \{2, \ldots, 6\}$ (tương ứng với các tầng từ $P_2 \rightarrow P_6$).
- Nhóm tác giả đã sử dụng pre-trained model [4], cho phép quá trình inference (suy luận) hiệu quả hơn nhưng chỉ tạo ra một lượng thụ phí tính toán nhỏ (small computational overhead).

Chương 5. RetinaFace Source Code

5.1 Dataset:

```
        for label in labels:
            annotation = np.zeros((1, 15))
            # bbox (x1, x2, y1, y2)
            annotation[0, 0] = label[0] # x1
            annotation[0, 1] = label[1] # y1
            annotation[0, 2] = label[0] + label[2] # x1 + w -> x2
            annotation[0, 3] = label[1] + label[3] # y1 + h -> y2

            # Landmarks - With repo data format
            annotation[0, 4] = label[4]   # l0_x
            annotation[0, 5] = label[5]   # l0_y
            annotation[0, 6] = label[7]   # l1_x
            annotation[0, 7] = label[8]   # l1_y
            annotation[0, 8] = label[10]   # l2_x
            annotation[0, 9] = label[11]   # l2_y
            annotation[0, 10] = label[13] # l3_x
            annotation[0, 11] = label[14] # l3_y
            annotation[0, 12] = label[16] # l4_x
            annotation[0, 13] = label[17] # l4_y

            if label[4] >= 0:
                annotation[0, 14] = 1
            else:
                annotation[0, 14] = -1
```

– Điểm chú ý nhất của source code dataset chính là cách tạo **annotation:**

+ Mỗi face được lưu thành vector bao gồm các label được khởi tạo. Tổng cộng có 4 giá trị bounding box (x_1, y_1, x_2, y_2), 10 landmark $(l_{0_x}, l_{0_y}, \ldots, l_{4_x}, l_{4_y})$ và một flag annotation[0, 14].

+ Label[6, 9, 12, 15] là vị trí của **visibility** và các giá trị này được bỏ qua.

+ Một face trong dataset có dạng:

```
    [x, y, w, h,
    l0_x, l0_y, v0,
    l1_x, l1_y, v1,
    l2_x, l2_y, v2,
    l3_x, l3_y, v3,
    l4_x, l4_y, v4]

```

+ Trong đó:

  * (l_{0_x}, l_{0_y}): left eye
  * l_{1_x}, l_{1_y}: right eye
  * l_{2_x}, l_{2_y}: nose
  * l_{3_x}, l_{3_y}: left mouth
  * l_{4_x}, l_{4_y}: right mouth

+ Visibility flag annotation[0, 14] = 1: landmarks hợp lệ, annotation[0, 14] = -1: landmarks bị missing.

5.2 Backbone:

– Paper đã sử dụng ResNet làm backbone, để trích xuất đặc trưng tạo thành các feature maps trước khi được đưa qua Context Module

– Tham khảo mã nguồn:

https://github.com/yuuMQ/Retinaface_FaceDetection_Pytorch/blob/main/models/backbones/resnet.py

```
        self.layer1 = self._make_layer(block, 64, layers[0])
        self.layer2 = self._make_layer(block, 128, layers[1],
    stride=2, dilate=replace_stride_with_dilation[0])
        self.layer3 = self._make_layer(block, 256, layers[2],
    stride=2, dilate=replace_stride_with_dilation[1])
        self.layer4 = self._make_layer(block, 512, layers[3],
    stride=2, dilate=replace_stride_with_dilation[2])


```

– Mỗi layer của ResNet là 1 Stage, trong đó chứa nhiều Residual Blocks.

– Ví dụ quá trình forward:

```
        input x: (1, 3, 224, 224)
            1. conv1 -> bn -> relu -> maxpool
                output: (1, 64, 56, 56)
            2. Layer1:
                (1, 64 * expansion, 56, 56)
                - BasicBlock -> (1, 64, 56, 56)
                - Bottleneck -> (1, 256, 56, 56)
            3. Layer2:
                (1, 128 * expansion, 28, 28)
            4. Layer3:
                (1, 256 * expansion, 14, 14)
            5. Layer4:
                (1, 512 * expansion, 7, 7)
            6. AvgPool:
                (1, 512 * expansion, 1, 1)
            7. Flatten + Fc:
                (1, num_classes)

```

– Tổng số layers tương ứng với mỗi phiên bản Resnet:

Model	**Layers (Số lượng block)**	**Tổng số Layers**
ResNet-18	[2, 2, 2, 2]	18
ResNet-34	[3, 4, 6, 3]	32
ResNet-50	[3, 4, 6, 3]	50
ResNet-101	[3, 4, 23, 3]	101
ResNet-152	[3, 8, 36,, 3]	152

5.3 Layers:

– **FPN (Feature Pyramid Network):**

+ FPN được triển khai cho multi-scale feature map extraction và mergin.

+ Sử dụng **1 × 1 convolutions** cho output layers và **3 × 3 convolutions** cho merging layers.

```
# 1x1 Convolution output layers
self.output1 = Conv2dNormActivation(in_channels_list[0],
out_channels, kernel_size=1, negative_slope=leaky)
self.output2 = Conv2dNormActivation(in_channels_list[1],
out_channels, kernel_size=1, negative_slope=leaky)
self.output3 = Conv2dNormActivation(in_channels_list[2],
out_channels, kernel_size=1, negative_slope=leaky)

# Merge Layers with 3x3 Convolutions
self.merge1 = Conv2dNormActivation(out_channels,
out_channels, kernel_size=3, negative_slope=leaky)
self.merge2 = Conv2dNormActivation(out_channels,
out_channels, kernel_size=3, negative_slope=leaky)

```

+ Quá trình forward:

```
    '''
    :param inputs: input feature maps from diff levels of the pyramid
    :return: list of merged output feature maps at diff scales.
    '''

    inputs = list(inputs.values())

    # Apply output layers to each feature map
    output1 = self.output1(inputs[0])
    output2 = self.output2(inputs[1])
    output3 = self.output3(inputs[2])

    # Merge outputs with upsampling and addition
    upsample3 = F.interpolate(output3, size=output2.shape[2:], mode='nearest')
    output2 = self.merge2(output2 + upsample3)

    upsample2 = F.interpolate(output2, size=output1.shape[2:], mode='nearest')
    output1 = self.merge1(output1 + upsample2)

    return [output1, output2, output3]


```

– **SSH (Single Stage Headless):**

+ SSH module sử dụng cho feature extraction bằng cách kết hợp $\{3 \times 3, 5 \times 5, 7 \times 7\}$ convolutions với batch normalization và activation function (LeakyReLU)

+ Thực chất đó là sự xếp chồng các convolution 3×3 để mô phỏng $5 \times 5, 7 \times 7$ convolution layers, giúp cho network nhẹ hơn nhưng vẫn bao quát được không gian xung quanh khuôn mặt.

```
    # 3x3 Convolution branch
    self.conv3X3 = Conv2dNormActivation(in_channel, out_channel
   // 2, kernel_size=3, activation_layer=None)

    # 5x5 Convolution branch
    self.conv5X5_1 = Conv2dNormActivation(in_channel,
   out_channel // 4, kernel_size=3, negative_slope=leaky)
    self.conv5X5_2 = Conv2dNormActivation(out_channel // 4,
   out_channel // 4, kernel_size=3, activation_layer=None)

    # 7x7 Convolution branch
    self.conv7X7_2 = Conv2dNormActivation(out_channel // 4,
   out_channel // 4, kernel_size=3, negative_slope=leaky)
    self.conv7X7_3 = Conv2dNormActivation(out_channel // 4,
   out_channel // 4, kernel_size=3, activation_layer=None)

```

5.4 Loss:

– Với công thức Multi-task loss:

$$L = L_{cls}(p_i, p_i^*) + \lambda_1 p_i^* L_{box}(t_i, t_i^*) + \lambda_2 p_i^* L_{pts}(l_i, l_i^*) + \lambda_3 p_i^* L_{pixel}. \tag{5.1}$$

ta có:

+ L_{cls}: Classification Loss L_{conf}

+ L_{box}: Face box regression Loss, trong source code, loss này được tính dựa trên sai số tọa độ của bounding box, được gọi là **Localization Loss** L_{loc}

+ L_{pts}: Facial landmark regression Loss F_{landm}, sai số của 5 facial feature tương ứng với 10 tọa độ (x, y)

```
    # Landmark Loss (Smooth L1)
    landm_p = landmark_preds[pos_idx1].view(-1, 10)
    lnm_targets = lnm_targets[pos_idx1].view(-1, 10)
    loss_landm = F.smooth_l1_loss(landm_p, lnm_targets,
   reduction='sum')

```

```
    # Localization Loss (Smooth L1)
    pos_idx = pos_mask.unsqueeze(pos_mask.dim()).expand_as(
   loc_preds)
    loc_p = loc_preds[pos_idx].view(-1, 4)
    loc_targets = loc_targets[pos_idx].view(-1, 4)
    loc_loss = F.smooth_l1_loss(loc_p, loc_targets, reduction='
   sum')

```

```
    # Confidence Loss Including Positive and Negative Examples
    pos_idx = pos_mask.unsqueeze(2).expand_as(conf_preds)
    neg_idx = neg_mask.unsqueeze(2).expand_as(conf_preds)
    conf_p = conf_preds[(pos_idx + neg_idx).gt(0)].view(-1,
   num_classes)
    targets_weighted = conf_targets[(pos_mask+neg_mask).gt(0)]
    conf_loss = F.cross_entropy(conf_p, targets_weighted,
   reduction='sum')

```

```
# Sum of losses: L(x,c,l,g) = (Lconf(x, c) + Lloc(x,l,g)) / N
    N = max(num_pos.sum().float(), 1)
    loc_loss /= N
    conf_loss /= N
    loss_landm /= N1

    return loc_loss, conf_loss, loss_landm

```

5.5 Optimizer:

– RetinaFace sử dụng SGD **(Stochastic Gradient Descent)** Optimizer:

$$\theta = \theta - \eta \nabla_\theta J(\theta; x_i, y_i) \tag{5.2}$$

– Sự khác biệt giữa **Gradient Descent** và **Stochastic Gradient Descent**

	Gradient Descent	**Stochastic Gradient Descent**
Công thức	$\theta = \theta - \eta \cdot \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta J_i(\theta)$	$\theta = \theta - \eta \cdot \nabla_\theta J_i(\theta)$
Dữ liệu mỗi step	Sử dụng **toàn bộ** tập huấn luyện (n mẫu).	Sử dụng duy nhất **1 mẫu** dữ liệu ngẫu nhiên.
Tốc độ hội tụ	Chậm (do phải tính toán trên toàn bộ tập dữ liệu).	Nhanh (cập nhật ngay lập tức sau mỗi mẫu).
Đường tối ưu	Mượt mà, đi thẳng đến cực tiểu.	Zic-zac, dao động mạnh xung quanh cực tiểu.
Thoát Local Minimum	Kém, dễ bị kẹt tại các hố nông.	Tốt hơn nhờ sự dao động (nhiễu) giúp nhảy khỏi hố.

5.6 Model:

– Sử dụng backbone (ResNet) để trích xuất đặc trưng (feature extraction).

– FPN: Kết hợp thông tin ngữ nghĩa từ các deep layer với thông tin không gian từ shallow layer.

– SSH: Tăng cường receptive field thông qua các convolution layer xếp chồng.

– Multi-task Heads: Thực hiện dự đoán song song trên các feature maps.

```
backbone = build_backbone(cfg['name'], cfg['pretrain'])
self.fx = get_layer_extractor(cfg, backbone) # feature extraction

num_anchors = 2
base_in_channels = cfg['in_channel']
out_channels = cfg['out_channel']

fpn_in_channels = [
    base_in_channels * 2,
    base_in_channels * 4,
    base_in_channels * 8,
]

self.fpn = FPN(fpn_in_channels, out_channels)
self.ssh1 = SSH(out_channels, out_channels)
self.ssh2 = SSH(out_channels, out_channels)
self.ssh3 = SSH(out_channels, out_channels)

self.class_head = ClassHead(in_channels=cfg['out_channel'], num_anchors=num_anchors, fpn_num=3)
self.bbox_head = BboxHead(in_channels=cfg['out_channel'], num_anchors=num_anchors, fpn_num=3)
self.landmark_head = LandmarkHead(in_channels=cfg['out_channel'], num_anchors=num_anchors, fpn_num=3)
```

– Quá trình forward:

```
    out = self.fx(x)
    fpn = self.fpn(out)

    # single-stage headless module
    feature1 = self.ssh1(fpn[0])
    feature2 = self.ssh2(fpn[1])
    feature3 = self.ssh3(fpn[2])

    features = [feature1, feature2, feature3]

    classifications = self.class_head(features)
    bbox_regressions = self.bbox_head(features)
    landmark_regressions = self.landmark_head(features)

    if self.training:
        output = (bbox_regressions, classifications,
   landmark_regressions)
    else:
        output = (bbox_regressions, F.softmax(classifications,
   dim=-1), landmark_regressions)
    return output
```

Chương 6. Ý tưởng cải tiến:

6.1 Hạn chế của paper hiện tại:

6.2 Swin Transformer for feature extraction:

6.3 Feature Enhancement Feature Pyramid Network:

Chương 7. Kết quả thực nghiệm:

Kết quả trên **WiderFace Evaluation Set** theo báo cáo của repository

– Multi-scale image resizing:

RetinaFace Backbones	**Easy**	**Medium**	**Hard**
MobileNetV1	90.59%	89.14%	84.13%
MoblieNetV2	91.70%	91.03%	86.60%
ResNet18	92.50%	91.02%	86.63%
ResNet34	94.16%	93.12%	88.90%

– Original image size:

RetinaFace Backbones	**Easy**	**Medium**	**Hard**
MobileNetV1	92.19%	90.41%	79.56%
MoblieNetV2	94.04%	92.26%	83.59%
ResNet18	94.28%	92.69%	82.95%
ResNet34	95.07%	93.48%	84.40%

Chương 8. Kết luận

REFERENCES

[1] S. Yang, P. Luo, C.-C. Loy, and X. Tang, "Wider face: A face detection benchmark," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 5525–5533, 2016.

[2] J. Dai, H. Qi, Y. Xiong, Y. Li, G. Zhang, H. Hu, and Y. Wei, "Deformable convolutional networks," in *Proceedings of the IEEE international conference on computer vision*, pp. 764–773, 2017.

[3] R. Girshick, "Fast r-cnn," in *Proceedings of the IEEE international conference on computer vision*, pp. 1440–1448, 2015.

[4] Y. Zhou, J. Deng, I. Kotsia, and S. Zafeiriou, "Dense 3d face decoding over 2500fps: Joint texture & shape convolutional mesh decoders," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 1097–1106, 2019.

[5] A. Ranjan, T. Bolkart, S. Sanyal, and M. J. Black, "Generating 3d faces using convolutional mesh autoencoders," in *Proceedings of the European conference on computer vision (ECCV)*, pp. 704–720, 2018.

[6] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," *Advances in neural information processing systems*, vol. 29, 2016.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[8] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 249–256, JMLR Workshop and Conference Proceedings, 2010.